DEPLOYMENT PLAN

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Content

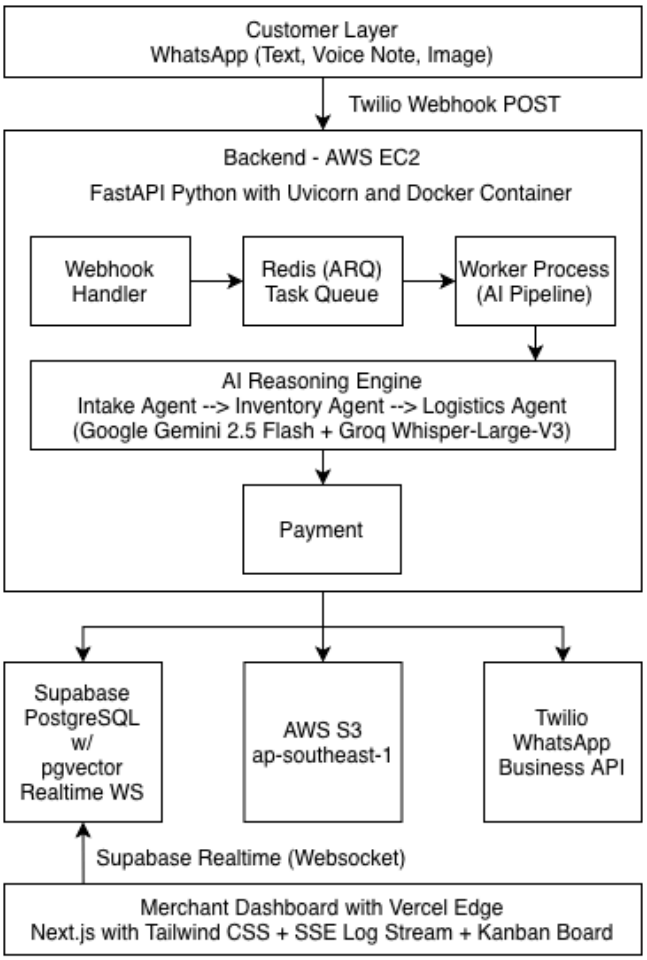
1. System Architecture Overview 2
2. State & Queue Management 4
3. CI/CD Pipeline & Automation 5
4. Integration Security & Data Privacy 5
5. Release Strategy & Reliability Framework 6
6. Scalability Strategy & Architectural Evolution 7
7. Financials and Maintenance Operations 8

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. System Architecture Overview

SupplyLah is designed as a **Cloud-Native, event-driven system** following a layered microservices pattern. Each concern, whether it is AI reasoning, media processing, real-time state and user interface, is isolated behind its own service boundary to allow independent scaling and technology swapping post-MVP.



© UMHackathon 2026

Email: umhackathon@um.edu.my

Component Registry

Layer	Technology	Version	Hosting
Frontend	Next.js (App Router, SSR)	15.1.3	Vercel Edge Network
Backend API	FastAPI + Uvicorn	0.115.5 / 0.32.1	AWS EC2 ap-southeast-1
Background Worker	ARQ (asyncio task queue)	>= 0.26.1	Co-located on EC2
Message Broker	Redis + hiredis	>= 5.0.0	EC2 (MVP) → AWS Elasticache (Scale)
Database	Supabase Managed PostgreSQL + pgvector	2.10.0 SDK	Supabase Cloud
LLM Reasoning	Google Gemini 2.5 Flash	-	Google AI Studio
Audio Transcription	Groq Whisper-large-v3	groq 1.2.0	Groq Cloud API
Media Storage	AWS S3	boto3 1.35.85	supplylah-media, ap-southeast-1
WhatsApp Gateway	Twilio WhatsApp Business API	Twilio 9.3.7	Twilio Cloud

AI Reasoning Engine with 3-Tier Hybrid Search

The product resolution pipeline avoids pure LLM-based matching by using a deterministic cascade before invoking the vector model, where the raw customer text moves to Tier 1 of ILIKE substring match on product_name, which takes roughly 0ms. If missed, then it moves to Tier 2, which uses the slang_aliases text array to perform alias containment which takes about 1ms. If missed, it moves to Tier 3, which performs pgvector cosine similarity via match_products RPC which takes about 15 to 30 ms.

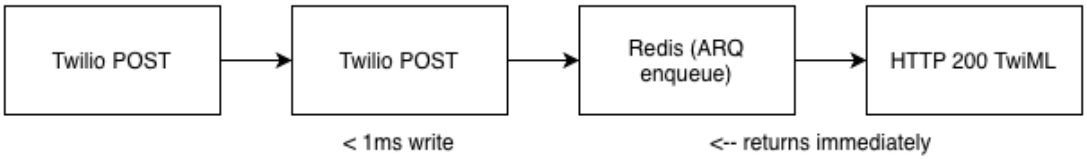
This design helps to keep 80% of the orders to be resolved without the vector index, preserving the Supabase free-tier RPC limits for edge cases such as ayam kampung to free-range chicken.

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. State & Queue Management

WhatsApp via Twilio will enforce a less than 5 second response SLA, so breaching it will cause Twilio to redeliver the webhook, creating duplicate orders. Our architecture prevents this by decoupling the receipt from processing:



The ARQ worker will pick up the job asynchronously which will invoke the intake agent, inventory agent and then logistics agent and finally send back to the customer via Twilio.

ARQ Worker Configuration

Parameter	Value	Justification
max_jobs	20	Concurrent AI jobs per worker process
job_timeout	300s	Covers full multi-agent pipeline including retries
max_tries	3	Auto-retry on Gemini or network failures
keep_result	3600s	1 hour result TTL (Time-to-Live) in Redis for observability debugging

Real-Time Dashboard Updates

The merchant dashboard subscribes to order state changes via Supabase Realitime Websockets with eventsPerSecond set to 10. Internally, the SSE (Server-Sent-Events) stream at

© UMHackathon 2026

Email: umhackathon@um.edu.my

/api/session-logs/{session_id} surfaces per-step AI reasoning logs to the MockChat panel using the asyncio.Queue per session.

3. CI/CD Pipeline & Automation

Current MVP State

Both a backend Dockerfile and a frontend/Dockerfile exist in the repository alongside a root-level docker-compose.yml for local orchestration. GitHub actions workflows are defined as the immediate next step post-hackathon, where the pipeline design below reflects the target production state.

Target Pipeline Architecture

The developer pushes the branch to the GitHub Actions for Continuous Integration such as Lint and Type Check, Pytest to validate LLM JSON outputs against Pydantic schemas and Docker build verification. After PR is merged to the main branch, GitHub Actions for Continuous Delivery will do deployment of Vercel CLI for frontend, which previews URLs per PR. Then for backend, SSH is done to EC2 as well as performing health-checks of the new image.

Key CI Validation Strategy

The most critical CI gate is the Pydantic schema conformance testing for LLM outputs. Each agent in the pipeline, the Intake, Inventory, Logistics, returns the structured JSON that is validated against a Pydantic model. CI runs a series of fixture messages to verify the model updates or prompt changes do not break the schema contract before reaching production.

4. Integration Security & Data Privacy

- **Secrets Management:** All sensitive credentials, including LLM API keys and Supabase service keys, are managed via encrypted environment variables and will migrate to AWS Secrets Manager with IAM role-based access to ensure quarterly rotation and eliminate static keys in the repository.
- **Database Row-Level Security (RLS):** Strict tenant isolation is enforced at the PostgreSQL engine level using Supabase RLS, guaranteeing that Wholesaler A can never access the inventory, teams, or order records of Wholesaler B.
- **Secure Webhook Authentication:** The system is designed to validate HMAC signatures on every inbound Twilio POST request, ensuring that only authenticated messages from official WhatsApp channels can trigger the agentic workflow.

© UMHackathon 2026

- **Restricted CORS Policy:** Cross-Origin Resource Sharing (CORS) is configured to allow interactions only from authorized Vercel production domains, effectively preventing unauthorized third-party frontend applications from calling the backend API.
- **PDPA Compliance and Media Security:** To align with Malaysia's Personal Data Protection Act (PDPA), customer voice notes and order images are stored in private AWS S3 buckets and are only accessible via server-generated, time-limited pre-signed URLs.
- **Automated Data Lifecycle:** The platform implements a 30-day automated deletion policy for all media assets, while ensuring that sensitive voice data is processed entirely in-memory and never stored on local server disks to maintain maximum data privacy.
- **Secure API Execution:** All privileged database operations and RPCs, such as atomic stock deductions, are examined with high-security definitions and are only callable by the backend orchestrator using a valid service-role key.

5. Release Strategy & Reliability Framework

SupplyLah utilizes a decoupled release architecture to minimize integration risks and ensure zero-downtime operations for wholesalers.

Runtime Feature Toggles

High-risk integrations such as Twilio and Lalamove are controlled by boolean flags in the `config.py` layer. This allows the team to hot-swap between mock environments and live production APIs instantly via environment variables without requiring code redeployments.

Grayscale Rollout (5% Pilot)

We follow a four-phase transition to ensure linguistic accuracy. Starting from the Hackathon MVP, we move to a Phase 1 pilot involving 1 to 2 wet market operators to monitor "Bahasa Rojak" parsing confidence. We expand to a 5% grayscale rollout only after achieving 90% parsing success threshold, leading to full regional availability.

Emergency Rollback

We implement a multi-tier rollback strategy to maintain the less-than-3-minute recovery target required for supply chain stability. This involves Blue-Green deployment patterns, where we re-pin the Docker image to a previous stable SHA in the Redis/ARQ queue, while using Vercel's instant rollback for the Command Centre frontend.

© UMHackathon 2026

Email: umhackathon@um.edu.my

Quality Gates

We utilize a low_confidence_threshold, with a default of 0.65 as an automated guardrail. Any order that does not meet this threshold is sent to a human-in-the-loop queue, preventing AI hallucinations from affecting real-world trade records.

6. Scalability Strategy & Architectural Evolution

To meet the demands of Malaysia's high-volume wholesale orders, SupplyLah is designed with a scalable growth roadmap. We have made deliberate architectural trade-offs to balance MVP development speed with production-grade reliability.

Current Scalability (Pilot Phase):
We currently utilize FastAPI with Asynchronous ARQ Workers. This allows the system to process heavy multimodal tasks in the background. If 100 customers send voice notes simultaneously, the system places them in a Redis Queue, ensuring the WhatsApp interface remains responsive and never hangs for the user.

The Scalability Trade-off (Monolith to Microservices):
For the MVP, we utilize a Containerized Monolith on AWS EC2. While a full Microservices architecture (AWS EKS) offers infinite scaling, it also brings high operational complexity. By choosing a Monolith now, we traded infinite scale for development speed, allowing us to refine our GLM reasoning faster. However, because we use Docker, our system is microservice-ready, allowing us to lift-and-shift each agent into its own independent scaling unit within days.

Database and Concurrency:
We use Supabase Connection Pooling to manage simultaneous data writes from hundreds of concurrent negotiation loops. As the system scales to Tier-1 wholesalers, we will transition from vertical scaling to horizontal scaling to maintain real-time dashboard performance.

Infrastructure Efficiency:
We trade higher per-call API costs such as Gemini and Groq for zero maintenance overhead. This allows us to scale our user base without needing a large DevOps team, keeping our maintenance fees low for the SME end-user.

© UMHackathon 2026

Email: umhackathon@um.edu.my

7. Financials and Maintenance Operations

7.1 Operational Expenditure (OpEx) Model

Category	Component / Provider	Basis of Estimate	Est. Monthly Cost (RM)
Cloud Infrastructure	AWS (EC2, S3, ALB)	API hosting, Redis worker, & 50GB media storage.	RM 1,200
AI Reasoning (LLM)	Gemini & Groq Whisper	~3 reasoning calls/order + ASR transcription.	RM 3,500
Data & Messaging	Supabase & Twilio	Managed PostgreSQL & per-conversation WhatsApp fees.	RM 2,450
Ops & Monitoring	Vercel & CloudWatch	Frontend edge hosting & log aggregation.	RM 400
TOTAL OpEx		**150 Wholesaler Pilot**	**RM 7,550**
Per Merchant Cost		**RM 7,550/150 merchants**	*≈ RM50/merchant/ month*

7.2 Maintenance and Reliability Framework

To protect the long-term integrity of the agentic workflow and ensure the system evolves with the merchant's needs, we implement a four-pillar maintenance strategy:

- **Prompt Engineering Lifecycle:** System prompts are separated from the core code and managed as versioned Markdown files. This allows for rapid iteration of "Bahasa Rojak" linguistic models and product aliases via simple database updates without requiring full system redeployments.
- **Agnostic LLM Hot-Swapping:** The architecture utilizes an OpenAI-compatible wrapper, making the agent loop model-agnostic. This enables the replacement or upgrade of the reasoning brain via environment variables with zero downtime.

© UMHackathon 2026

Email: umhackathon@um.edu.my

- **Proactive Security and Patching:** We ensure system resilience through weekly automated Docker image rebuilds and Dependabot monitoring for Python dependency vulnerabilities, while Supabase manages all backend database patching as a managed service.
- **Operational Observability:** The system utilizes structured logging and real-time SSE (Server-Sent Events) streams to provide a transparent audit trail of the AI's reasoning steps. This allows for high-frequency monitoring and immediate post-mortem debugging of any failed order states.
- **Sustainability & Maintenance Fee:** To cover these ongoing optimization costs, an annual maintenance fee of RM 500 is applied per organization. This ensures the agent remains compliant with local regulations, such as LHDN e-invoicing update, and continues to accurately process regional slang.

© UMHackathon 2026

Email: umhackathon@um.edu.my